

SpendWise

Secure Expense Tracker

Track daily spending, budgets and savings goals,
with every account private and every change logged.

~220 MB	219	5 tries	0
RAM in use	automated tests	then a 15 min lockout	servers to install

Java 17 - Spring Boot 3.5 - Spring Security - Thymeleaf - JPA - H2

Project guide: what it does, how to set it up, and how it works

github.com/BhonePyaeMin/secure-expense-tracker

OVERVIEW

About the project

SpendWise is a personal finance web app I built to track daily spending on a low-spec laptop. Each user has a private account protected by BCrypt-hashed passwords, CSRF protection, and a lockout after five failed logins. Every query is scoped to the signed-in user, and every change is written to an audit log.

It goes beyond a basic CRUD tracker: monthly budgets with over-budget warnings, a daily allowance, recurring bills, savings goals, spending insights, a trash with restore, and backup and restore through CSV files. It runs in about 220 MB of RAM using an embedded H2 database, with no Docker and no database server.

What you can do with it

Log spending in seconds

Add, edit and search expenses with a category, date, payment method and note. Quick add understands "lunch 85 baht yesterday", the category is guessed from the title, and Repeat copies an expense to today.

Stay inside a budget

Set a monthly limit per category; over-budget rows turn red. The daily allowance divides what is left by the days left in the month, and income gives a monthly balance.

Understand your habits

Insights compare each category with last month, project the month-end total, and show the top 5 expenses and spending by weekday. Savings goals show progress and how much to put aside each month.

Never lose data

Deleted expenses go to a Trash first. Backup downloads a ZIP of CSV files; Restore previews it, skips bad rows with a reason, and adds or replaces your data only after you confirm.

STEP 1 - SETUP

Set it up in five minutes

The only requirement is a Java 17 (or newer) JDK. Maven is not needed because the project ships with the Maven wrapper, and there is no database server or Docker to install: the data lives in a single H2 file.

1. Get the code

```
git clone https://github.com/BhonePyaeMin/secure-expense-tracker.git
cd secure-expense-tracker
```

2. Start the app

```
./mvnw spring-boot:run
```

The first run downloads the dependencies (a few minutes); later starts take about 5 seconds. On Windows the same command works in PowerShell; in the old cmd.exe type mvnw instead of ./mvnw.

3. Open it and create your account

```
http://localhost:8080
```

Register a username and a password (8 to 72 characters). Your data is saved in ./data/expenses.mv.db and survives restarts.

Low-memory run (about 220 MB)

```
./run.sh            # macOS / Linux / Git Bash
.\run.bat           # Windows
```

The scripts build the jar the first time, then start it with -Xmx256m -Xms64m -XX:+UseSerialGC -XX:TieredStopAtLevel=1. After changing the code, rebuild with ./mvnw -DskipTests package.

Try it with sample data, or run the tests

```
./mvnw spring-boot:run -Pdemo     # sign in as demo / demo1234
./mvnw test                       # 219 tests, in-memory database
```

Profiles

Profile	Start with	What it does
default	`./mvnw spring-boot:run`	Your real data in ./data/expenses; H2 console off
dev	`./mvnw spring-boot:run -Pdev`	Adds the H2 console at /h2-console (only after signing in)
demo	`./mvnw spring-boot:run -Pdemo`	About 26 sample expenses in a separate file, ./data/demo
test	`./mvnw test`	Fresh in-memory database for every test run

STEP 2 - USING IT

Everyday workflow

1

Sign up and sign in

Create an account; passwords are stored only as BCrypt hashes. Five wrong passwords in a row lock the account for 15 minutes, and every sign-in and failed attempt is recorded.

2

Log what you spend

Add an expense with the form or type it in plain words with Quick add. Filter by month and category, search titles and notes, export the current view to CSV, or import a bank CSV that is checked row by row first.

3

Set budgets and add income

Give each category a monthly limit and record your income, so the app can show a balance and warn you before you overspend.

4

Check the summary every day

Totals per category and per payment method, a spending-by-day chart, budget bars, and today's allowance: the amount you can still spend per day for the rest of the month.

5

Review insights and goals

See which categories grew compared with last month, where the month is heading, your biggest expenses, and how your savings goals are progressing.

6

Let recurring bills run themselves

Rent and subscriptions are added automatically every day at 00:05 when they are due, with catch-up if the laptop was off and a database rule that prevents duplicates.

7

Clean up, back up, restore

Deleted expenses wait in the Trash with Undo. Download a backup at any time and restore it later with a preview, a skipped-rows report, and Add or Replace.

STEP 3 - ARCHITECTURE

How it works inside

A classic three-layer Spring MVC application in a single process. Pages are rendered on the server with Thymeleaf, with a little plain JavaScript for Quick add, the category hint and the chart.



Every request passes through each layer in order; each layer only talks to the next one.

One request, step by step: saving an expense

1 Browser
The form sends POST /expenses with a hidden CSRF token.

2 Spring Security
Checks the session and the CSRF token. Not signed in: redirect to the sign-in page. Bad token: 403.

3 ExpenseController
Binds the fields to an ExpenseForm and validates them (amount above 0, at most 2 decimals, date not in the future). On errors it shows the form again with your input kept. The owner id comes from the session, never from the form.

4 ExpenseService
Builds the Expense, saves it, and writes an EXPENSE_CREATED entry to the audit log.

5 ExpenseRepository
Spring Data JPA turns the save into SQL on the H2 file. Every query includes the owner id and leaves the trash out.

6 Back to the browser
Redirect to /expenses with "Expense added." (post / redirect / get, so a refresh never saves twice).

Rules that hold everywhere

Money is BigDecimal

Never double. Every calculated amount goes through one Money helper: 2 decimal places, rounded HALF_UP.

One clock

"Today" always comes from a single Asia/Bangkok Clock bean, so a late-night expense lands on the right day.

Schema without migrations

Hibernate keeps the tables up to date (ddl-auto=update); new columns are nullable so old rows stay valid. Expenses are indexed on (owner, date), date, category, and (date, category).

STEP 4 - SECURITY

Security by design

Threat	How SpendWise handles it
Password theft	BCrypt hashes only; passwords 8 to 72 characters; the hash is erased from the session after sign-in.
Password guessing	5 wrong passwords in a row lock the account for 15 minutes (stored in the database). Wrong username and wrong password show the same message.
Seeing other users' data	Every lookup includes the owner id. Another user's expense returns 404, exactly like a missing one, so ids cannot be probed.
CSRF	A token on every POST form, including sign-out; HttpOnly, SameSite=Lax session cookie.
SQL injection	Only Spring Data queries with parameters; % and _ in searches are escaped.
XSS	Thymeleaf escapes all output; chart data goes through data-* attributes.
CSV injection	Exported cells starting with = + - @ are prefixed so spreadsheets don't run them.
Leaking details	Error pages never show stack traces or messages; the log never contains amounts, notes or passwords.
Hostile uploads	1 MB upload limit, 2,000-row imports, 5 MB unzip cap on restore, all-or-nothing import.
Secrets	No passwords or keys in the config files; a database password comes from DB_PASSWORD.
Accountability	An activity log of sign-ins, failures, lockouts and every change with before and after values.

STEP 5 - RESOURCES

Built for a low-spec laptop

The whole app runs in about 220 MB of process memory and starts in about 5 seconds. That comes from a handful of deliberate choices:

Small JVM

-Xmx256m caps the heap, the Serial GC is the lightest collector, and TieredStopAtLevel=1 trades peak speed for lower memory and faster startup.

Embedded database

H2 in file mode: no database server, no Docker, one file to back up.

Lean Spring

Lazy bean initialization, 20 web threads, a connection pool of 5, open-in-view off, and no Actuator. The only dependencies beyond the Spring Boot starters are H2 and spring-security-test.

STEP 6 - QUALITY

Tested and checked

219 automated tests (JUnit 5, MockMvc, DataJpaTest, Spring Security Test) run on every push with GitHub Actions. They cover the money and projection math, user isolation, CSRF, lockout, recurring catch-up, import and export, a backup-then-restore round trip, error pages that leak nothing, a log that never contains private values, and "today" around midnight in Bangkok. Every page was also checked at 360 px wide: no sideways scrolling and 44 px tap targets.

Tech stack

Java 17 · Spring Boot 3.5 · Spring Security · Thymeleaf · Spring Data JPA · H2 Database · BCrypt · Chart.js · JUnit 5 · Maven Wrapper · GitHub Actions

Source: github.com/BhonePyaeMin/secure-expense-tracker - Built by Bhone Pyae Min